

پروژه دوره پایتون پیشرفته آکادمی همراه اول

قصد ما در این بخش از درس، فراتر رفتن از مباحث نظری و ورود به یک تجربه عملی و سازنده است. می‌خواهیم با هم دانشی که در طول ترم آموخته‌ایم را در قالب یک پروژه واقعی به کار بگیریم و یک نرم‌افزار کوچک اما کامل را از ابتدا تا انتها توسعه دهیم. این پروژه نه تنها به شما کمک می‌کند تا درک عمیق‌تری از ابزارها و تکنیک‌های برنامه‌نویسی پیدا کنید، بلکه تجربه کار بر روی یک محصول نرم‌افزاری را نیز برایتان به ارمغان می‌آورد. پروژه‌ای که برای شما در نظر گرفته‌ام، ساخت یک "سیستم مدیریت یادداشت شخصی" است.

تشریح پروژه: سیستم مدیریت یادداشت شخصی

بیاید با هم نگاه دقیق‌تری به این پروژه و اجزای مختلف آن بیندازیم:

۱. مفهوم و عملکرد اصلی

"سیستم مدیریت یادداشت شخصی" همانطور که از نامش پیداست، ابزاری دیجیتال برای کمک به کاربران در ثبت، سازماندهی و بازیابی یادداشت‌هایشان خواهد بود. فکر کنید به یک دفترچه یادداشت هوشمند که همیشه و همه‌جا در دسترس است.

- **عملیات اصلی (CRUD):** قلب تپنده سیستم شما، چهار عملیات بنیادین خواهد بود:
 - **ایجاد (Create):** کاربر باید بتواند یک یادداشت جدید با عنوان و محتوای دلخواه اضافه کند. در این مرحله، سیستم شما باید به طور خودکار یک شناسه یکتا (ID) به یادداشت اختصاص دهد و تاریخ و زمان ایجاد را نیز ثبت نماید.
 - **خواندن (Read):** کاربران باید قادر باشند لیست خلاصه‌ای از تمام یادداشت‌ها (مثلاً فقط ID و عنوان) را مشاهده کنند، و همچنین بتوانند با انتخاب یک یادداشت خاص، تمام جزئیات آن (ID، عنوان، محتوا، تاریخ ایجاد و آخرین ویرایش) را ببینند.
 - **به‌روزرسانی (Update):** این قابلیت به کاربر اجازه می‌دهد عنوان یا محتوای یک یادداشت از قبل ایجاد شده را ویرایش کند. پس از هر ویرایش، تاریخ "آخرین ویرایش" یادداشت نیز باید به‌روز شود.
 - **حذف (Delete):** کاربر باید بتواند یادداشتی که دیگر به آن نیاز ندارد را به طور کامل از سیستم حذف کند.

۲. مدیریت و ذخیره‌سازی داده‌ها

نحوه نگهداری اطلاعات در این پروژه در دو سطح تعریف می‌شود:

- **ذخیره‌سازی در حافظه (In-memory):** برای شروع و سادگی کار، ابتدا یادداشتهای را در یک لیست پایتونی (لیستی از اشیاء یا دیکشنری‌ها) در حافظه اصلی برنامه نگهداری خواهید کرد. این روش به شما کمک می‌کند تا بدون درگیر شدن با پیچیدگی‌های فایل سیستم یا دیتابیس، بر روی منطق اصلی برنامه و عملکرد صحیح عملیات CRUD تمرکز کنید. البته، با این روش، با هر بار بستن و اجرای مجدد برنامه، اطلاعات از بین خواهند رفت.
- **پایداری داده‌ها با JSON (بخش تکمیلی و اختیاری):** برای اینکه یادداشتهای کاربران با ارزش باشند و با بسته شدن برنامه از دست نروند، شما یک مکانیزم برای ذخیره دائمی آن‌ها پیاده‌سازی خواهید کرد. این کار با ذخیره کردن لیست کامل یادداشتهای در یک فایل با فرمت JSON (مثلاً notes.json) انجام می‌شود. هنگام شروع برنامه، یادداشتهای از این فایل خوانده شده و در حافظه بارگذاری می‌شوند و هنگام خروج (یا پس از هر تغییر مهم)، اطلاعات به‌روز شده مجدداً در فایل JSON ذخیره می‌گردند.
 - **ساختار فایل JSON:** این فایل معمولاً حاوی یک آرایه (list) از اشیاء (objects) خواهد بود که هر شیء، نماینده یک یادداشت با فیلدهای مشخصی مانند id, title, content, created_at, updated_at است.
 - **عملیات فایل:** شما نیاز به توابعی برای خواندن داده‌ها از فایل و تبدیل آن‌ها به اشیاء پایتونی، و برعکس، تبدیل اشیاء پایتونی به فرمت JSON و نوشتن آن‌ها در فایل خواهید داشت. مدیریت خطاهای احتمالی هنگام کار با فایل (مانند عدم وجود فایل، خرابی فرمت، یا مشکلات دسترسی) بخش مهمی از این قسمت است.

۳. رابط‌های کاربری

کاربران باید بتوانند از دو طریق با سیستم شما تعامل داشته باشند:

- الف. رابط خط فرمان (CLI) با Typer:

یک رابط خط فرمان کارآمد و واضح، بخش مهمی از این پروژه است.

پروژه دوره پایتون پیشرفته آکادمی همراه اول

- note create: پس از اجرای این دستور، برنامه باید به صورت تعاملی، ابتدا عنوان و سپس محتوای یادداشت را از کاربر درخواست کند. برای ورود محتوا، به امکان ورود متن‌های چندخطی توجه داشته باشید. پس از ایجاد موفق، پیامی حاوی شناسه (ID) یادداشت جدید به کاربر نمایش داده شود.
- note list: خروجی این دستور باید لیستی خوانا از یادداشت‌ها، شامل ID و عنوان هر یک باشد تا کاربر بتواند به سرعت یادداشت مورد نظر خود را بیابد.
- note show <id>: با دریافت ID یک یادداشت، تمام جزئیات آن (عنوان، محتوا، تاریخ‌ها) به شکلی منظم نمایش داده شود. در صورت نامعتبر بودن ID، پیام خطای مناسبی به کاربر داده شود.
- note update <id>: این دستور می‌تواند ابتدا جزئیات فعلی یادداشت را نمایش داده، سپس از کاربر بپرسد که قصد ویرایش کدام بخش (عنوان یا محتوا) را دارد و مقدار جدید را دریافت کند.
- note delete <id>: پیش از حذف نهایی، توصیه می‌شود یک پیام تأیید از کاربر دریافت کنید (مثلاً: "آیا از حذف یادداشت با ID X مطمئن هستید؟ [y/N]").
- ب. رابط کاربری وب (Web UI) با FastAPI و Jinja2:

همان طور که در درس گفته شد، FastAPI به همراه Uvicorn برای نوشتن web-application ها و restful API ها به کار می‌رود. FastAPI می‌تواند خروجی یک مسیر (route) را یک JSON و یا text و یا HTML خروجی دهد. در این پروژه تمرکز را بر روی HTML می‌گذاریم.

برای همین شما نیازمند دانش حداقلی به ساختار HTML و CSS و JS دارید. سایت w3schools مرجع خوبی برای این موضوع است که می‌توانید به آن مراجعه کنید. برای این موضوع نیاز داریم از یک template engine استفاده کنیم. خود FastAPI از Jinja2 پشتیبانی می‌کند. حتماً کار با این template engine را یاد بگیرید.

- **صفحات وب مورد نیاز:** به صفحاتی مانند لیست نمایش یادداشت‌ها، فرم ایجاد یادداشت جدید، صفحه نمایش جزئیات یک یادداشت، و فرم ویرایش یادداشت نیاز خواهید داشت.
- **مسیرهای FastAPI (Routes):** برای هر عملیات CRUD و هر صفحه وب، مسیرهای مناسبی در FastAPI

پروژه دوره پایتون پیشرفته آکادمی همراه اول

تعریف خواهید کرد. به عنوان مثال، GET /notes برای لیست یادداشت‌ها، POST /notes برای ایجاد یادداشت جدید از طریق فرم وب، GET /notes/{note_id} برای نمایش یک یادداشت خاص، و غیره.

- استفاده از Jinja2: از قابلیت‌های Jinja2 مانند حلقه‌ها برای نمایش لیست‌ها، نمایش متغیرها در HTML، وراثت قالب‌ها (template inheritance) برای ایجاد یک ساختار پایه مشترک بین صفحات، و همچنین برای ساخت فرم‌های HTML استفاده خواهید کرد.

۴. جنبه‌های کلیدی توسعه و ارزیابی

اهداف این پروژه تمرین decorator ها و شی گرای است. موفقیت شما در این پروژه تنها به اجرای صحیح عملکردها خلاصه نمی‌شود، بلکه نحوه پیاده‌سازی و رعایت اصول مهندسی نرم‌افزار نیز بسیار حائز اهمیت است.

• شی‌گرایی (OOP):

- انتظار می‌رود یک کلاس Note با صفات (attributes) مناسب مانند id, title, content, creation_date, last_modified_date و متدهای کاربردی طراحی کنید. به کپسوله‌سازی و مسئولیت‌های هر کلاس دقت کنید.
- پیاده‌سازی یک کلاس NoteManager (یا نامی مشابه) که مسئولیت مدیریت مجموعه یادداشت‌ها (افزودن، یافتن، حذف، ذخیره/بارگذاری) را بر عهده بگیرد، می‌تواند به ساختارمند شدن کد شما کمک شایانی کند.

• Decorator ها:

- شما به طور طبیعی از دکوریتورهای ارائه شده توسط FastAPI (مانند app.get@ و Typer (مانند app.command@) استفاده خواهید کرد. علاوه بر این، می‌توانید برای درک بهتر، دکوریتورهای سفارشی ساده‌ای نیز بنویسید، مثلاً دکوریتوری برای لاگ کردن زمان اجرای یک تابع خاص یا یک دکوریتور برای مدیریت یک نوع خطای خاص در چند تابع.

• Logging:

- استفاده صحیح از ماژول logging برای ثبت وقایع مهم در سطوح مختلف (مانند INFO برای عملیات موفق، ERROR برای خطاها، و DEBUG برای اطلاعات دقیق اشکال‌زدایی)

پروژه دوره پایتون پیشرفته آکادمی همراه اول

بسیار مهم است. این لاگ‌ها به شما در فهم رفتار برنامه و رفع مشکلات کمک خواهند کرد.

- **pytest:**

- نوشتن تست‌های واحد برای منطق اصلی (business logic) برنامه الزامی است. این تست‌ها باید مستقل از رابط کاربری باشند و صحت عملکرد کلاس‌ها و توابع اصلی شما را (مانند افزودن یادداشت، جستجوی یادداشت، ذخیره در JSON) تأیید کنند.
- **Typser:** همانطور که پیشتر گفته شد، برای ساخت رابط خط فرمان از این ابزار استفاده خواهید کرد.
- برای ساختار پروژه، از ساختار تمیز معرفی شده در درس، یعنی با استفاده از **cookiecutter** (یا الگوبرداری از ساختار استاندارد آن)، استفاده کنید.
- این به معنای سازماندهی کد در ماژول‌ها و فایل‌های جداگانه با مسئولیت‌های مشخص است. به عنوان مثال، می‌توانید ماژول‌هایی برای مدل‌ها (**models.py**)، عملیات ذخیره‌سازی (**storage.py**)، منطق اصلی سرویس‌ها (**services.py** یا **crud.py**)، مسیرهای وب (**/routers**)، دستورات CLI و قالب‌ها (**note/resources/templates**) داشته باشید.

۵. راهنمایی:

در هنگام استفاده از **cookiecutter** معرفی شده در درس، می‌توانید **layout** را روی **src** و یا **flat** انتخاب کنید. در کلاس درس با استفاده از **src layout** به صورت پیش فرض انجام دادیم. در اینجا بهتر است از ساختار **flat** استفاده کنید. شباهت‌ها و تفاوت‌های این دو ساختار را در اینترنت جست و جو کنید و مطالعه کنید.

اسم ماژول را **note** بگذارید. برای بحث **cli** نویسی، یک فایل در پوشه **note** به اسم **__main__.py** بسازید. این فایلی است که موقع **python -m note** صدا زده می‌شود. دستورات مربوط با **cli** را با استفاده از **typer** در این فایل بنویسید.

همان گونه که گفته شد دستورات **cli** مانند **create** و **delete** و ... باشد. یک دستور دیگر به نام **web** بسازید. **Web application** را با این دستور اجرا کنید. **Port** را نیز به عنوان یک متغیر آپشنال به این

پروژه دوره پایتون پیشرفته آکادمی همراه اول



دستور وارد کنید. دستور `python -m note --help` باید خروجی ای مانند زیر داشته باشد.

```
Usage: python -m note [OPTIONS] COMMAND [ARGS]...
```

```
Personal Note Manager - Manage your notes efficiently
```

Options

```
--help      Show this message and exit.
```

Commands

```
create      Create a new note interactively.
delete      Delete a note.
list        List all notes.
search      Search notes by title or content.
show        Show details of a specific note.
update      Update an existing note.
web         Run the web application.
```

```
D:\Desk\project\hamrah academy\project\note-manager> python -m note list
No notes found
D:\Desk\project\hamrah academy\project\note-manager> python -m note create
Creating a new note
Enter note title: my test
Enter note content (press Ctrl+C when finished):
this is a test
2025-06-01 12:24:47,388 - note.storage - INFO - Successfully saved 1 notes to notes.json
2025-06-01 12:24:47,390 - note.services - INFO - Created new note: Note(id=e85e5c9e..., title='my test')
✓ Note created successfully with ID: e85e5c9e-173b-49d2-80d2-a9ca78d7ff5d
D:\Desk\project\hamrah academy\project\note-manager> python -m note list
Your Notes
```

ID	Title	Created	Updated
e85e5c9e-173b-49d2-80d2-a9ca78d7ff5d	my test	2025-06-01 12:24	2025-06-01 12:24

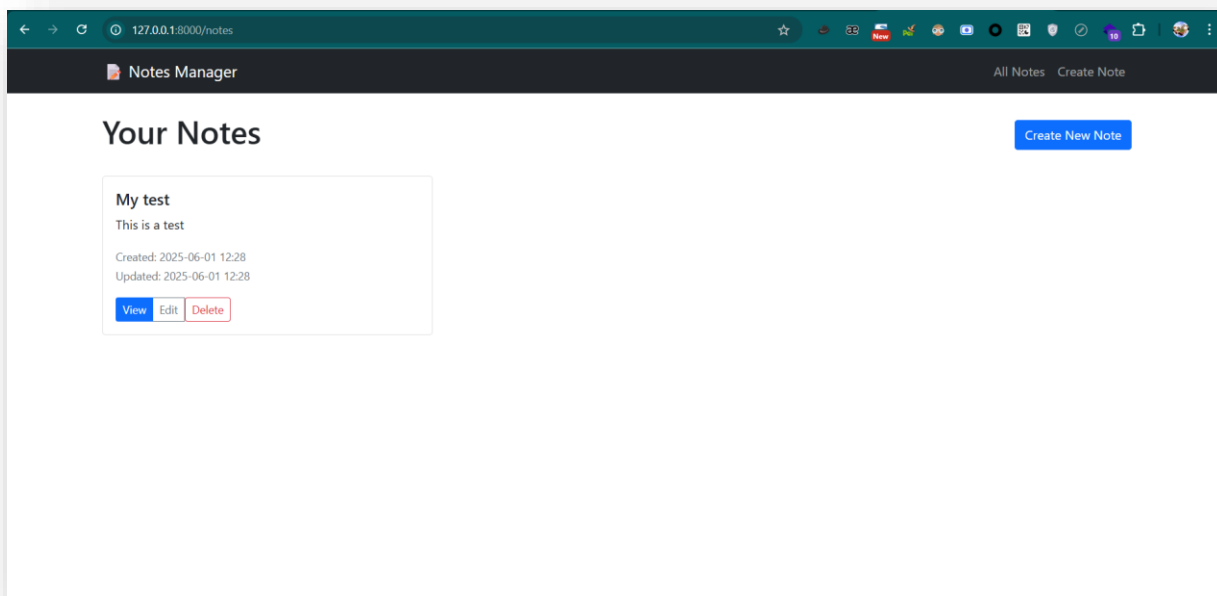
برای تست از `python -m pytest --cov` استفاده کنید. همچنین دو فایل `note/__main__.py` و `note/web_app.py` را در فایل `pyproject.toml` در بخش `[tool.coverage.run]` در `omit` جدا کنید تا در محاسبه `coverage` نقش نداشته باشد. نتیجه باید بدون هیچ خطایی و دارای `coverage` بالای ۸۰ درصد باشد.

پروژه دوره پایتون پیشرفته آکادمی همراه اول



برای این پروژه حق تغییر دادن tool.ruff.lint را در pyproject.toml را ندارید و باید تمام لینتر ruff بدون خط اجرا شود.

برای بخش webapplication از شما انتظار ui پیچیده و حرفه ای نیست (مانند خروجی زیر) صرفاً یک ui ساده برای بیان هدف کافیهست. هدف اصلی کار کردن با jinja2 و fastapi است.



همچنین route مسیر / را کاری کنید که به صورت خودکار روی /notes هدایت شود.

هر المان در notes.json یک id مشخص داشته باشد. برای این منظور میتوانید از کتابخانه uuid.uuid4 استفاده کنید. نمونه یک فایل notes.json در زیر آمده است.

```
[  
  {  
    "id": "971ed414-eae3-41ee-8990-3cc8f7d661f8",  
    "title": "My test",  
    "content": "This is a test\r\n",  
    "created_at": "2025-06-01T12:28:37.380412",  
    "updated_at": "2025-06-01T12:28:37.380412"  
  }  
]
```

همچنین خروجی log ها را در فایل note_manager.log قرار دهید.

امیدوارم این توضیحات جامع، مسیر روشنی را برای شروع و پیشبرد این پروژه در اختیار شما قرار دهد.
موفق باشید!

«لطفاً توجه داشته باشید:

- ✓ هدف اصلی این پروژه سنجش مهارت‌های شخصی شماست.
- ✓ استفاده از هوش مصنوعی برای حل کامل مسئله مجاز نیست.